



课程群二维码

Introduction To Database System

数据库引论

CS20010.01

上课： H2214

周三11~13节 第**1**~16周

上机： H逸夫楼204&205

周五01~02节 第**2**~16周

考试： 2026.7.1 13:00~15:00

闭卷

Teaching Information

▶ Instructor

- ▶ 卢瞰
- ▶ Email: lutun@fudan.edu.cn
- ▶ <http://cscw.fudan.edu.cn>
- ▶ 江湾校区学科交叉二号楼A2007

▶ Teaching Assistants

- ▶ 詹伟德: 25113050306@m.fudan.edu.cn
- ▶ 涂凯南: 25213050047@m.fudan.edu.cn

▶ Course Site

- ▶ Elearning系统: PPT和教辅
 - ▶ <https://elearning.fudan.edu.cn/courses/107343>



Semester Schedule

周次	日	一	二	三	四	五	六	备 注
0	22	23	24	25	26	27	28	9. 本科生线上申请补考, 2月25日至3月1日补考, 3月1日注册, 3月2日上课。
1	3/1	2	3	4	5	6	7	10. 研究生线上申请补考, 2月25日至3月1日补考, 2月27日注册, 3月2日上课。
2	8	9	10	11	12	13	14	11. 妇女节、清明节、劳动节、青年节及端午节放假以学校办通知为准。
3	15	16	17	18	19	20	21	12. 5月15日、16日第64届校田径运动会暨第6届教工趣味运动会, 5月15日停课一天。
4	22	23	24	25	26	27	28	13. 5月27日建校121周年, 开展校庆学术活动等。
5	29	30	31	4/1	2	3	4	14. 通识教育课程考试安排在第16周, 第17、18周为停课考试周。
6	5	6	7	8	9	10	11	15. 2026届本科生、研究生毕业典礼于第18周举行。
7	12	13	14	15	16	17	18	16. 第二学期于2026年7月4日结束, 共计18教学周(包括考试)。
8	19	20	21	22	23	24	25	17. 2026年7月5日起开展各类本科生、研究生暑期教学等活动。
9	26	27	28	29	30	5/1	2	18. 研究生寒假、暑假时间由院系和导师根据培养计划妥善安排。
10	3	4	5	6	7	8	9	19. 教职工原则上每学期提前一周上班, 延后一周开始寒暑假轮休。具体时间安排由学校办另行通知。
11	10	11	12	13	14	15	16	
12	17	18	19	20	21	22	23	
13	24	25	26	27	28	29	30	
14	31	6/1	2	3	4	5	6	
15	7	8	9	10	11	12	13	
16	14	15	16	17	18	19	20	
17	21	22	23	24	25	26	27	
18	28	29	30	7/1	2	3	4	

■ 清明节

■ 4月4日(周六)~
4月6日(周一)无调休

■ 劳动节

■ 5月1日(周五)~
5月5日(周二)
■ 5月9日(周六)调休

■ 端午节

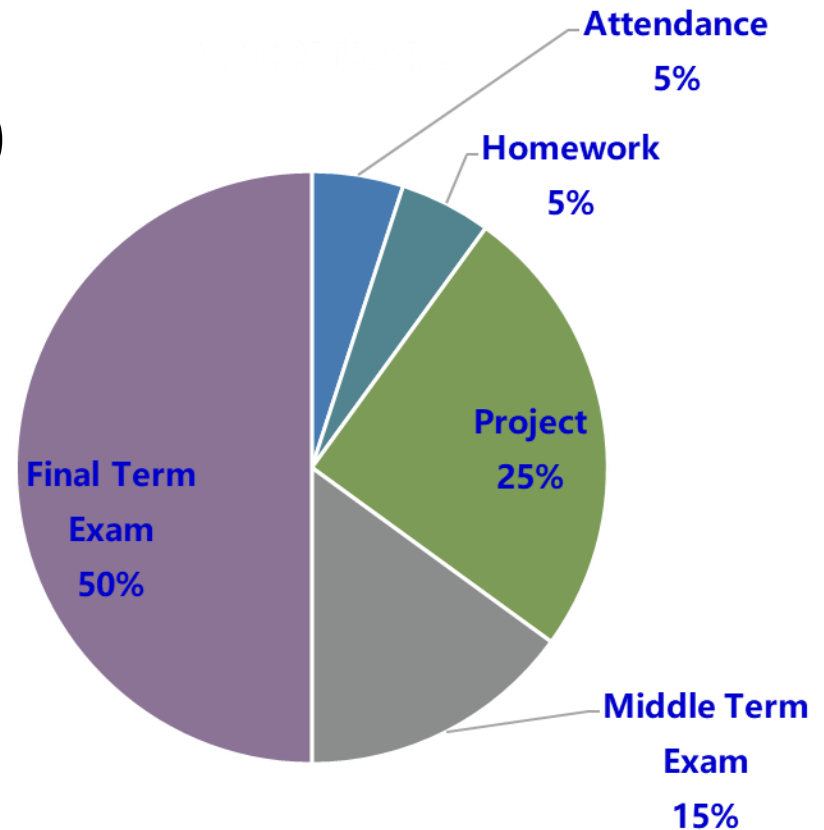
■ 6月19日(周五)~
6月21日(周日)无调休

停课
考试周



Grade Policy

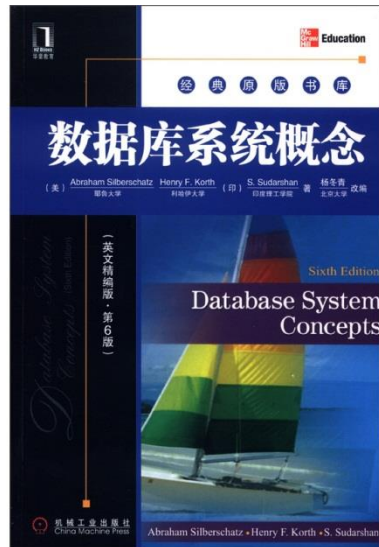
- ▶ Attendance (5%)
- ▶ Homework (5%)
- ▶ Project (25%)
- ▶ Middle Term Exam (15%)
- ▶ Final Term Exam (50%)



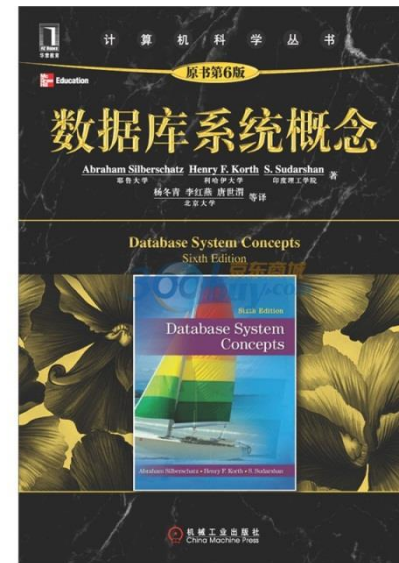
Textbook

- ▶ Textbook:

- ▶ **Database System Concepts** (6th edition),
Abraham Silberschatz, Henry F. Korth, S.Sudarshan.



English Edition



Chinese Edition

Textbook

- ▶ This textbook is used by many universities.
 - ▶ Stanford University
 - ▶ Yale University
 - ▶ University of Texas at Austin
 - ▶ University of Illinois at Urbana-Champaign
 - ▶



Reference

- Database Management Systems (3rd Edition), Raghu Ramakrishnan, Johannes Gehrke, Mcgraw Hill
- David M.Kronenke, Database Processing: Fundamentals, Design, and Implementation, Prentice Hall

Syllabus

- ◆ 第1周：数据库简介；关系数据库模型（数据库模式、键、模式图）；关系数据库模型（关系查询语言、关系操作）
- ◆ 第2周：SQL简介（SQL数据定义、SQL查询基本结构、集合操作、空值、聚集函数、嵌套子查询）；中级SQL（连接表达、视图、事务、完整性约束）；高级SQL（函数与过程、触发器）
- ◆ 第3周：形式化查询语言（关系代数）；第三节形式化查询语言（元组关系算子）
- ◆ 第4周：数据库设计与E-R模型（实体-关系模型、约束）；数据库设计与E-R模型（E-R图、模式约简）；数据库设计与E-R模型（E-R设计）
- ◆ 第5周：关系数据库设计（原子域与1NF、函数依赖）；关系数据库设计（BCNF）



Syllabus

- ◆ 第6周：关系数据库设计（3NF）；关系数据库设计（多值依赖、关系分解）；关系数据库设计（4NF、数据库设计过程与方法）
- ◆ 第7周：数据存储与文件系统（RAID、文件系统、记录的组织、数据库缓存）；索引与哈希（有序索引、B+树索引文件、多键值访问）
- ◆ 第8周：索引与哈希（静态哈希、动态哈希）；索引与哈希（位图索引）
- ◆ 第9周：查询处理（查询代价）；查询处理（选择操作）；查询处理（排序操作）
- ◆ 第10周：查询处理（连接等操作）；查询处理（查询表达式赋值）
- ◆ 第11周：查询优化（关系表达式转换）；查询优化（表达式结果的估计统计）；查询优化（查询计划的选择）
- ◆ 第12周：事务（事务的概念）；事务（ACID性质）；事务（序列化）



Syllabus

- ◆ 第13周：并发控制（基于锁的协议）；并发控制（死锁处理）；并发控制（多粒度锁）
- ◆ 第14周：并发控制（基于时间戳的协议）；并发控制（多版本模式）；并发控制（快照隔离）
- ◆ 第15周：系统恢复（失效分类）；系统恢复（恢复的原子性）
- ◆ 第16周：系统恢复（恢复算法、缓存管理）；系统恢复（缓存管理）



Main Contents

▶ **Part One: Relational Database**

- ▶ Introduction to the Relational Model
 - ▶ Structure of Relational Databases, Relational Query Languages.....
- ▶ Introduction to SQL
 - ▶ Basic SQL Operations, Set Operations, Aggregate Functions, Modification of the Database
- ▶ Intermediate SQL
 - ▶ Join Expressions, Views, Transactions, Integrity Constraints

Main Contents

- ▶ **Part One: Relational Database**
 - ▶ Advanced SQL
 - ▶ Functions and Procedures, Triggers
 - ▶ Formal Relational Query Languages
 - ▶ The Relational Algebra, The Tuple Relational Calculus, The Domain Relational Calculus

Main Contents

▶ **Part Two: Database Design**

- ▶ Database Design and the E-R Model
 - ▶ Entity-Relationship Model, Constraints,
- ▶ Relational Database Design
 - ▶ Atomic Domains and First Normal Form, Functional-Dependency Theory.....

Main Contents

- ▶ **Part Three: Data Storage and Query**
 - ▶ Storage and File Structure
 - ▶ Magnetic Disk and Flash Storage, File Organization,
 - ▶ Indexing and Hashing
 - ▶ Ordered Indices, Static Hashing, Dynamic Hashing....
 - ▶ Query Processing
 - ▶ Parsing and translation, Evaluation
 - ▶ Query Optimization

Main Contents

▶ **Part Four: Transaction Management**

▶ Transactions

- ▶ Transaction Concept, Transaction Atomicity, Durability and Isolation

▶ Concurrency Control

- ▶ Lock-Based Protocols, Deadlock Handling, Timestamp-Based Protocols.....

▶ Recovery System

- ▶ Failure Classification, Recovery Algorithm, Early Lock Release and Logical Undo Operations



Project

▶ **Four basic projects**

- ▶ 1. Basic operations of SQL Server
- ▶ 2. Design relational database using ER Model
- ▶ 3. Learn to create, delete and modify relational table using SQL statement.
- ▶ 4. Using ODBC to connect to database

▶ **Two advanced projects**

- ▶ Design a book sales management system
- ▶ Modify the source code of PostgreSQL to learn about the internal mechanism of database management system.

▶ Get started **EARLY!**

复旦数据库科研与教学的起源

施伯乐：开拓计算机数据库科研与教学之路



https://dl.ccf.org.cn/video/videoDetail.html?_ack=1&id=6310688397019136

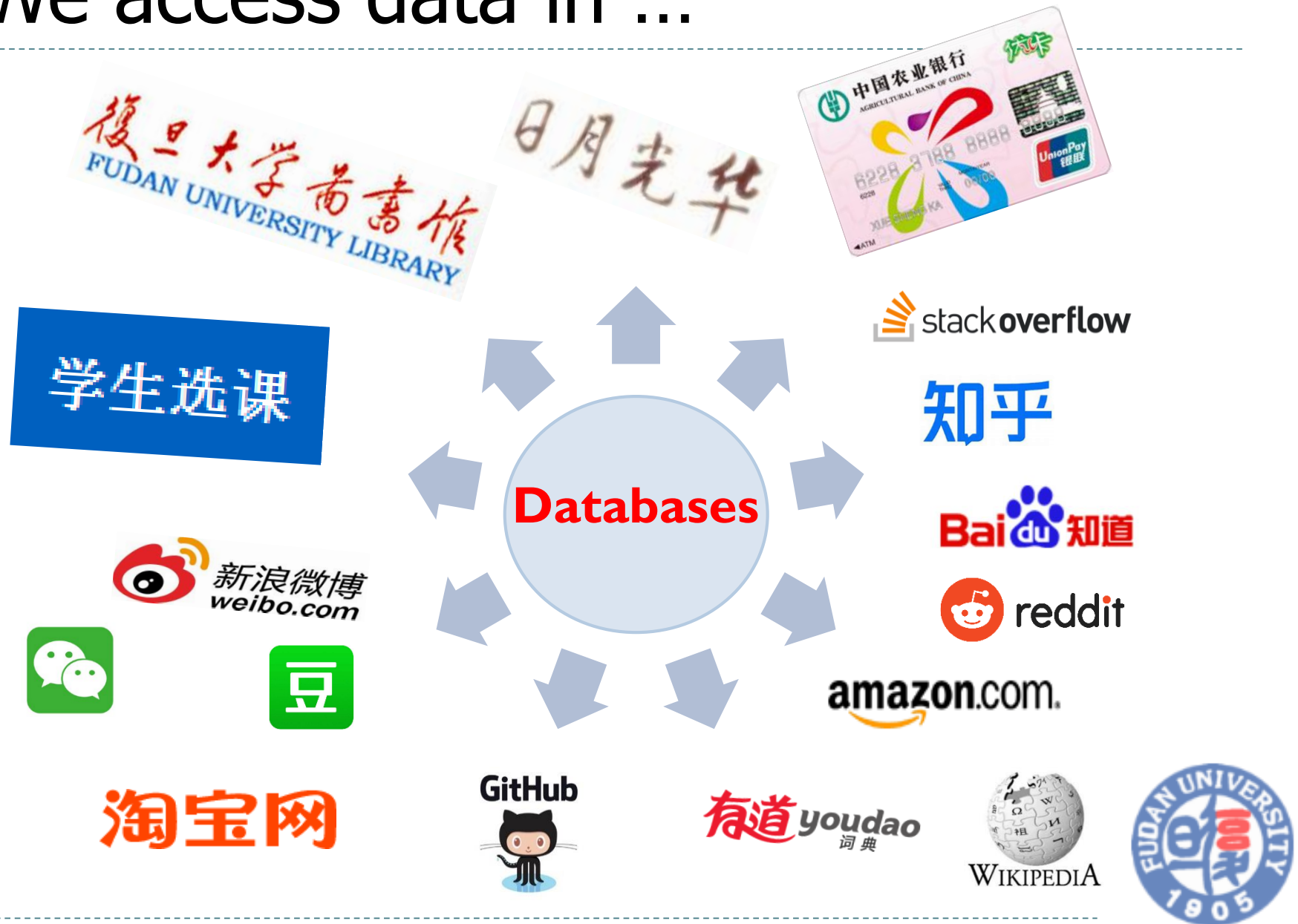


中国计算机事业60年
杰出贡献特别奖(2016)

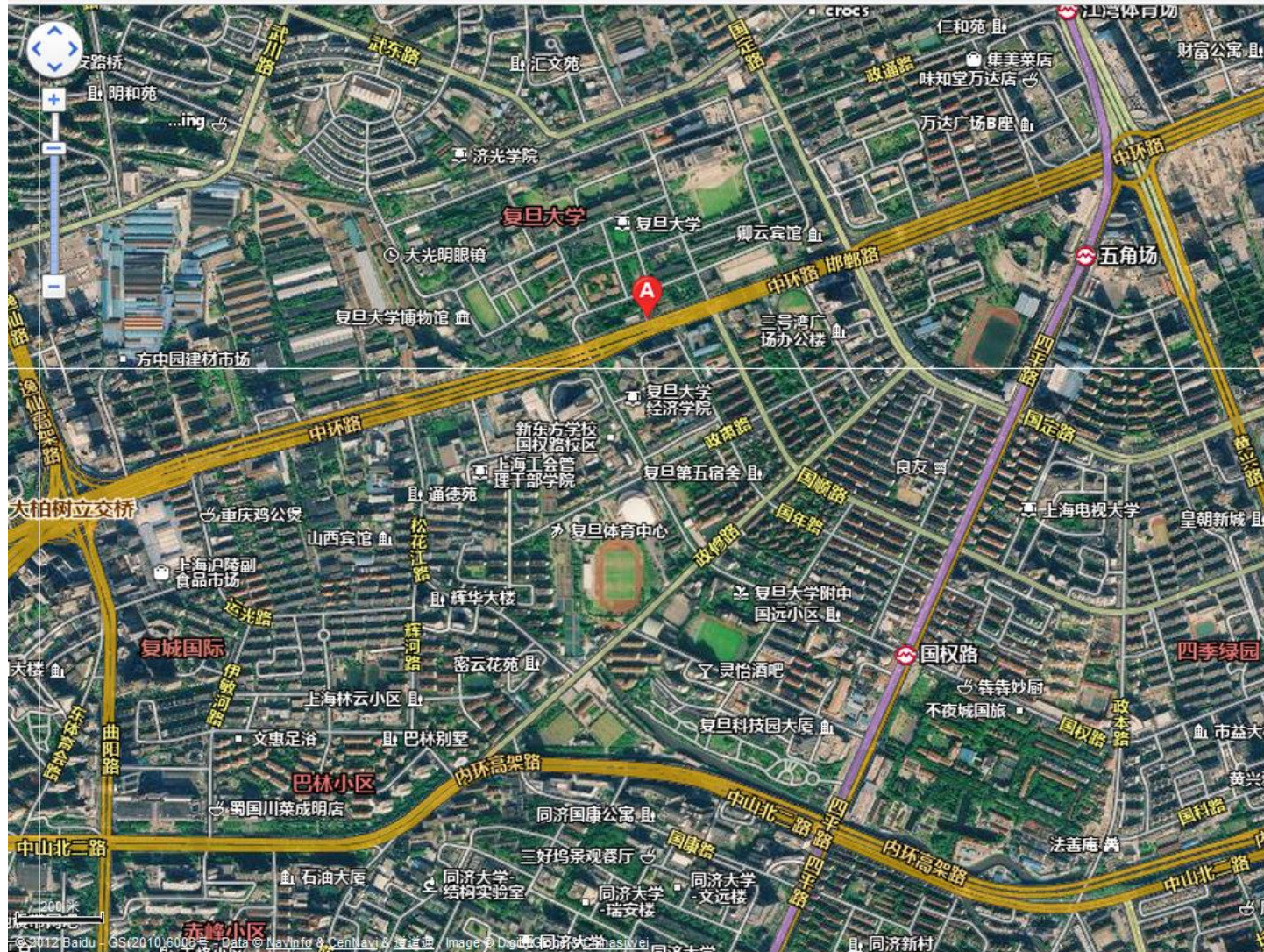
Chapter 1: Introduction



We access data in ...



Other Kinds of Databases (spatial)



Spatial



So...*What* Is a Database ?



- ▶ The amount of information available to us is exploding
- ▶ Users require tools that simplify the tasks of managing the data and extracting useful information
- ▶ A **Database** is collection of data: typically describing the activities of one or more related organizations



Database Management System (DBMS)

- ▶ **DBMS** contains information about a particular enterprise
 - ▶ Collection of interrelated data
 - ▶ Set of programs to access the data
 - ▶ An environment that is both *convenient* and *efficient* to use
- ▶ **Database Applications**
 - ▶ Banking: transactions
 - ▶ Airlines: reservations, schedules
 - ▶ Universities: registration, grades
 - ▶ Sales: customers, products, purchases
 - ▶ Online retailers: order tracking, customized recommendations
 - ▶ Manufacturing: production, inventory, orders, supply chain
 - ▶ Human resources: employee records, salaries, tax deductions
- ▶ Databases can be very large
- ▶ Databases touch all aspects of our lives



University Database Example

- ▶ **Application program** examples
 - ▶ Add new students, instructors, and courses
 - ▶ Register students for courses, and generate class rosters
 - ▶ Assign grades to students, compute grade point averages (GPA) and generate transcripts
- ▶ In the early days, database applications were built directly on top of *file systems*



History

- ▶ The first DBMS was designed by **Charles Bachman** at GE in early 1960s
 - ▶ *Network Data Model*
 - ▶ CODASYL (Conference on Data Systems Languages)
 - ▶ The *ACM's Turing award* in 1973
- ▶ In the late 1960s, IBM developed the Information Management System (IMS) DBMS
 - ▶ *Hierarchical data model*
- ▶ In 1970 **Edgar Codd** at IBM proposed **relational data model**
 - ▶ Codd won the *Turing award* in 1981



History (Cont.)

- ▶ The **SQL** query for relational databases, developed as part of IBM's **System R** project, was standardized in the late 1980s
 - ▶ The current standard, SQL-99, was adopted by ANSI (American National Standards Institute) and ISO (International Standards Organization)
- ▶ **Jim Gray** won the 1999 **Turing award** for his work on database transaction management
 - ▶ Born 12 January 1944, lost at sea 28 January 2007



History (Cont.)

- ▶ In the late 1980s and the 1990s, **advances** have been made in many areas of database systems
 - ▶ **Several vendors** (e.g., **IBM's DB2, Oracle, Informix UDS**) have extended their systems with the ability to store new data types such as images and text, and to ask more complex queries
 - ▶ **Specialized systems** have been developed by numerous vendors for creating **data warehouses**, consolidating data from several databases, and for carrying out specialized analysis



History (Cont.)

- ▶ An interesting phenomenon is the emergence of several **enterprise resource planning (ERP)** and **management resource planning (MRP)** packages
 - ▶ They add a substantial layer of **application-oriented** features on top of a DBMS
 - ▶ Widely used packages include systems from **Oracle**, **SAP**, etc.

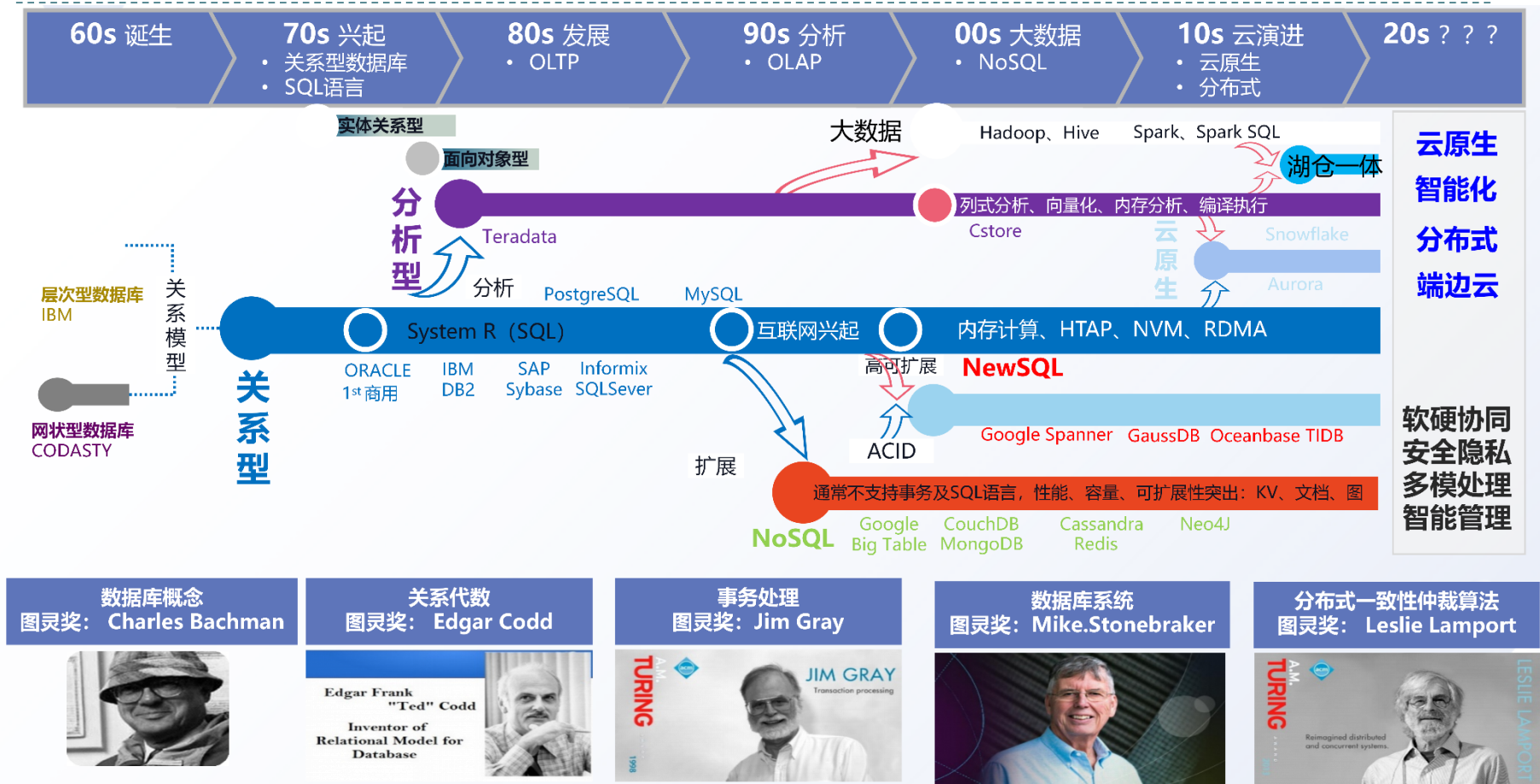


History (Cont.)

- ▶ DBMSs have entered the **Internet Age**
 - ▶ While the first generation of **Web sites** stored their data exclusively in operating systems files, the use of a DBMS to store data that is accessed through a **Web browser** is becoming widespread
 - ▶ Queries are generated through Web-accessible forms
 - ▶ Answers are formatted using a **markup language such as HTML, and Extensible Markup Language (XML)**, thus easily displayed in a browser

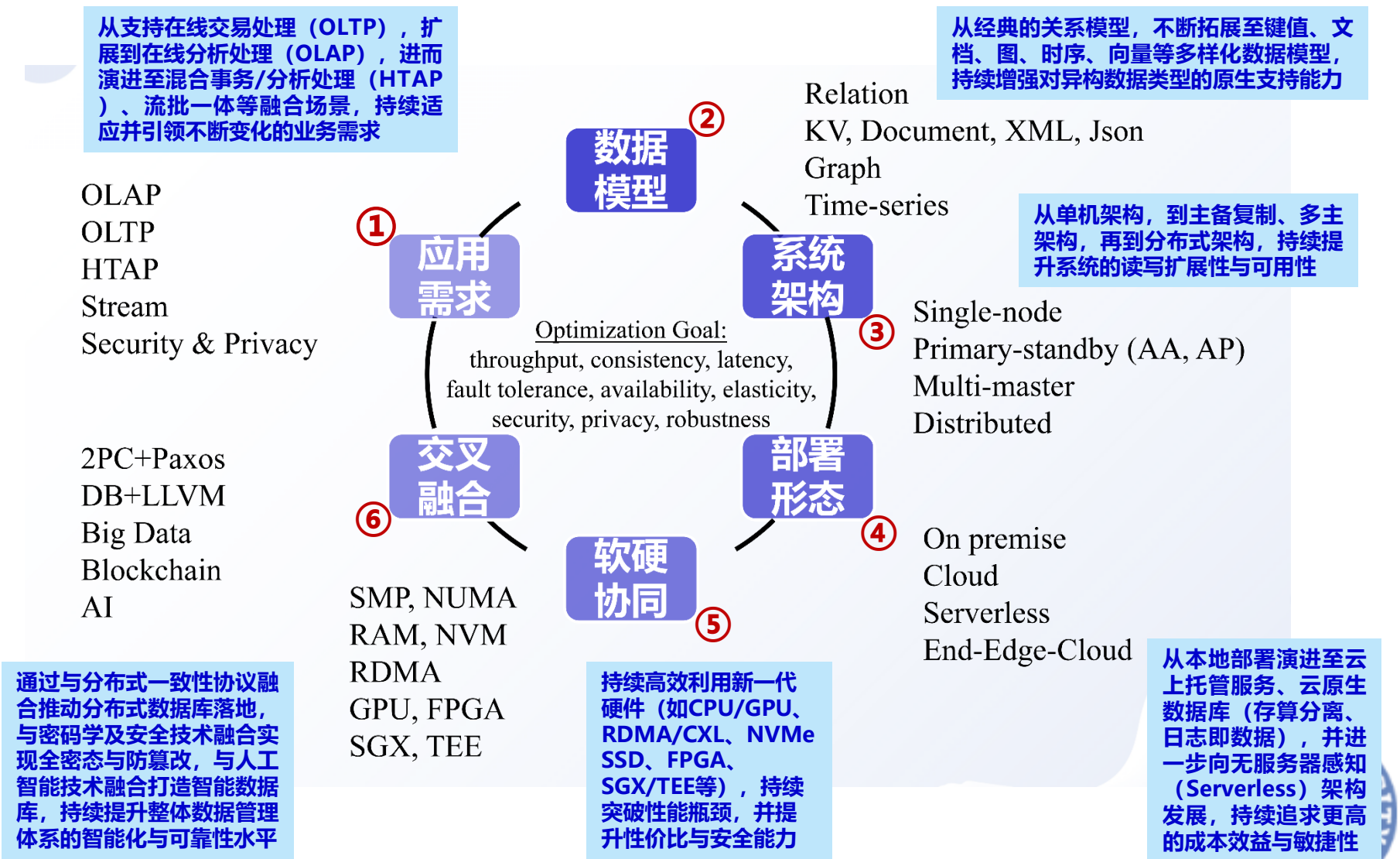


数据库的发展历程：简史



- 1970年代：关系模型兴起，奠定了关系数据库的理论基石；
- 1980年代：事务型数据库逐渐成熟，成为企业核心业务系统的支柱；
- 1990年代：分析型数据库与数据仓库崛起，赋能复杂的商业智能分析与决策；
- 2000年代：大数据与NoSQL数据库快速发展，以应对海量、多源异构数据的高可扩展存储与查询处理挑战；
- 2010年代：云原生数据库兴起，通过弹性伸缩与按需使用，重构了数据库的交付与运维模式；
- 2020年~：数据库迈入智能化新阶段，朝着多模态融合、自然语言交互、智能调优与主动服务的方向演进

数据库的发展历程：影响数据库发展的因素



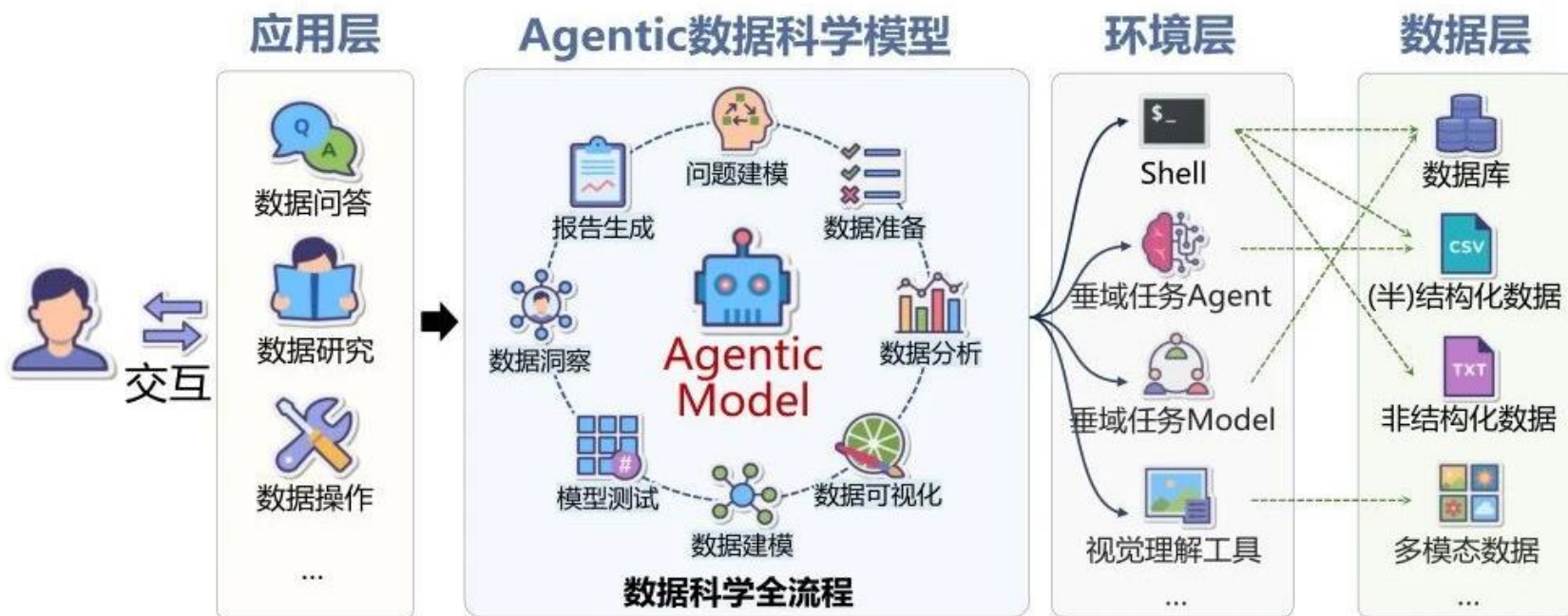
数据库的发展趋势：

自主数据系统（数据智能体）——AI/ Agent-Ready 数据库的未来形态

- **深度集成与赋能AI应用而设计的新一代数据管理系统**：从被动的数据存储库，转变为能够理解、推理并主动响应的高级数据服务平台
- **五个维度的根本性变革**
 - **数据模型（关系型→多模态融合）**：超越单一的结构化数据管理，实现对文档、向量、图片等多模态数据的原生融合与统一处理。这使其从静态的“数据存储库”升级为驱动AI应用的主动智能中枢，核心职责贯穿数据集成关联、智能发现探索，直至面向AI的高性能访问，提供统一集成、智能感知与即时响应的全方位能力
 - **查询模型（声明式查询→自然语言查询）**：支持使用自然语言直接进行查询、交互分析与报告生成，极大降低了使用门槛。其查询过程具备探索性（如交互式数据模式发现）与可重叠优化特性，标志着从需要精确语法的传统模式，向具备深度语义理解能力的智能交互的关键跨越
 - **计算范式（字符串匹配→语义理解与生成）**：实现从“字符串匹配”到“语义理解与生成”的根本转变，其核心在于内嵌的语义理解与推理引擎，能够解析用户意图，并提供一个支持混合检索（向量+全文+关系过滤）的高性能计算底座，以保障智能体 workflows 所需的实时响应与海量向量计算，优化目标也从单一追求低延迟，演进为降低LLM消耗、提升准确率、兼顾延迟的多目标平衡
 - **设计理念（被动式服务→主动式服务）**：实现从“被动响应”到“主动服务”的范式转移，服务主体从人转向AI与智能体，以应对其高频、复杂、模糊的请求，系统通过记忆管理、上下文感知、反馈优化与持续学习等机制，能够主动提供情境化的智能数据服务
 - **部署方式（静态预制部署→动态按需部署）**：拓展Serverless架构，部署方式正从静态预配置全面转向动态按需供给，这不仅实现了资源的快速启停、弹性伸缩与按需消费，更完成了从“预先规划资源”到“实时匹配需求”的根本性转变，从而达成了资源的高效动态适配与系统的持续自主优化
- **智能时代的数据基础设施新图景**：不再是沉默的后端系统，而是能对话、思考并主动协作的智能伙伴

数据库的发展趋势：自主数据库系统案例

自主数据科学系统



- 区别于传统手动、割裂的流程，这类系统能够**自主编排从数据发现到模型评估的完整数据科学工作流**，即：给定一个自然语言描述的任务（如“预测用户流失”），系统可自动将其分解为子任务管道，并调度不同的功能智能体协同完成，不需要用户手工写代码
 - **模块化的功能智能模块**：针对数据科学各环节（如数据发现、清洗、模型选择、训练评估）开发专业化智能体
 - **开放的工具体集成**：无缝连接外部机器学习库、特征工程工具及可视化平台等生态系统
 - **动态的管道编排与优化**：根据自然语言查询自动选择并调度智能体，通过动态优化确保流程效率与鲁棒性，形成可解释、自适应且持续演进的自动化工作流
 - **自闭环反馈系统**：系统能够基于执行状态与中间结果进行动态感知与自主调整，通过持续反馈循环，最终自动编排生成准确的执行计划，实现从感知、决策到执行的完整自闭环
 - **可扩展的架构**：设计允许通过规范接口集成新的功能智能体，在统一框架内实现无缝协作

Drawbacks of using file systems to store data

- ▶ **Data redundancy and inconsistency**

- ▶ Multiple file formats, duplication of information in different files

- ▶ **Difficulty in accessing data**

- ▶ Need to write a new program to carry out each new task

- ▶ **Data isolation** — multiple files and formats

- ▶ **Integrity problems**

- ▶ Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
- ▶ Hard to add new constraints or change existing ones



Drawbacks of using file systems to store data

▶ **Atomicity of updates**

- ▶ Failures may leave database in an inconsistent state with partial updates carried out
- ▶ Example: Transfer of funds from one account to another should either complete or not happen at all

▶ **Concurrent access by multiple users**

- ▶ Concurrent access needed for performance
- ▶ Uncontrolled concurrent accesses can lead to inconsistencies
- ▶ Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time

▶ **Security problems**

- ▶ Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems



Advantages of a DBMS

- ▶ **Data Independence**

- ▶ Application program should not be exposed to details of data representation and storage
- ▶ DBMS provides an abstract view of the data that hides such details

- ▶ **Efficient Data access**

- ▶ DBMS can support sophisticated techniques to store and retrieve data efficiently



Advantages of a DBMS (Cont.)

▶ **Data Integrity and Security**

- ▶ DBMS can enforce integrity constraints
 - ▶ E.g. check departmental budget before inserting salary information
- ▶ DBMS can enforce access controls that govern what data is visible to different classes of users

▶ **Data Administration**

- ▶ Provides facilities for
 - ▶ organizing the data representation to minimize redundancy
 - ▶ fine-tuning the storage of the data to make retrieval efficient



Advantages of a DBMS (Cont.)

- ▶ **Concurrent Access and Crash Recovery**

- ▶ DBMS schedules concurrent access such that users can think of the data as being accessed by only one user at a time
- ▶ Protects users from the effects of system failures

- ▶ **Reduced Application Development Time**

- ▶ DBMS supports important functions that are common to many applications
- ▶ High-level interface to facilitates quick application development



But In the Following Situations, we **DON'T** Use DBMS

- ▶ A DBMS is a complex piece of software, optimized for certain kinds of workloads , and its *performance may not be adequate* for certain specialized applications
- ▶ An application may need to manipulate the data in ways *not supported by the query language*
- ▶ If *specialized performance or data manipulation requirements are central* to an application, the application may choose not to use a DBMS



Data Description and Storage

▶ **Data Models**

- ▶ A **data model** is a collection of concepts for describing data, data relationships, and data constraints
- ▶ It is a high-level data description constructs that hide many low-level storage details

▶ **Schema**

- ▶ A **schema** is a description of a particular collection of data, using a given data model



Data Description and Storage

▶ **Semantic Data Model**

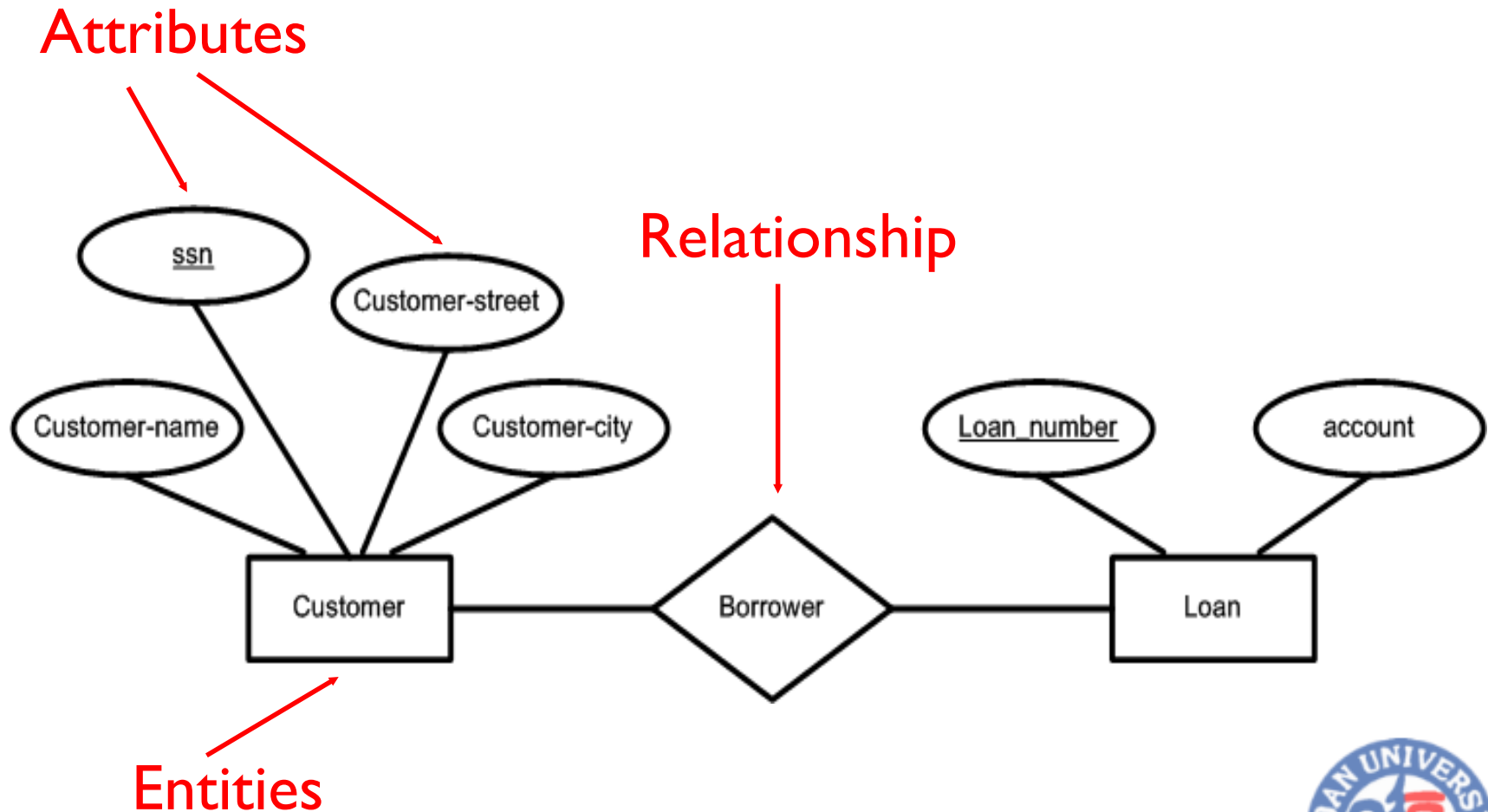
- ▶ A **Semantic data model** is a more abstract, high-level data model
- ▶ Easier for a user to come up with a good initial description of the data in enterprise

▶ **Entity-Relationship (ER) Model**

- ▶ A widely used semantic data model
- ▶ Allows us to pictorially denote entities and the relationships among them



E.g. An Example of ER Model



The Relational Model

- ▶ Most DBMS today are based on the relational data model
- ▶ **Relation**
 - ▶ *central data description construct* in this model
 - ▶ It can be thought of as *a set of records*
 - ▶ A table with *rows and columns*
 - ▶ Row – a record
 - ▶ Column – field, attribute



The Relational Model

Tuple or Row

Attribute or Column

sid	name	login	age	gpa
53666	Jones	Jones@cs	18	3.4
53688	Smith	Smith@ee	18	3.2
53650	Smith	Smith@math	19	3.8
53831	Madayan	Madayan@misic	11	1.8
53832	Guldu	guldu@music	12	2.0

An example of a student relation

- ▶ The **schema** of the above table is:

Students(*sid* : string, *name*: string, *login*: string,
age: integer, *gpa*: real)



Levels of Abstraction in a DBMS

Users



External Schema 1

External Schema 2

External Schema 3

Conceptual Schema

Physical Schema

DISK

Can have **Several**,
each tailored to a
particular group of
users

Just **One**



Several Data Models

- ▶ **Relational data model**: IBM's DB2, Informix, Oracle, Sybase, SQL Server, Access, FoxBase, Paradox, Tandem, and Teradata
- ▶ **Hierarchical model**: IBM's IMS DBMS
- ▶ **Network model**: IDS and IDMS
- ▶ **Object-oriented model**: Objectstore and Versant
- ▶ **Object-relational model**: used in DBMS products from IBM, Informix, ObjectStore, Oracle, Versant, UNI SQL and others



Conceptual Schema (Logical Schema)

- ▶ The **Conceptual Schema** describes *what* data should be stored in the database, and the *semantics* of the data
 - ▶ In terms of the data model of the DBMS
- ▶ In a relational DBMS, describes *all relations that are stored in the database*
 - ▶ Entity
 - ▶ Relationship



A sample University Database

Students(*sid*: string, *name*: string, *login*: string,
 age: integer, *gpa*: real)

Faculty(*fid*: string, *fname*: string, *sal*: real)

Courses(*cid*: string, *cname*: string, *credits*: integer)

Rooms(*rno*: integer, *address*: string, *capacity*: integer)

Enrolled(*sid*: string, *cid*: string, *grade*: string)

Teaches(*fid*: string, *cid*: string)

Meets_In(*cid*: string, *rno*: integer, *time*: string)

Entities: Students, Faculty, Courses, Rooms

Relationships: Enrolled, Teaches, Meets_In



Physical Schema

- ▶ The **Physical Schema** specifies *additional storage details*
 - ▶ *How* and *where* data are actually stored, low level data structures are specified at this level
 - ▶ File Organizations
 - ▶ Indexes



External Schema (View level)

- ▶ The **External Schema** allows *data access to be customized (and authorized)* for a particular type of users
- ▶ Each external schema consists of *a collection of one or more views and relations* from the conceptual schema
- ▶ A view is conceptually a relation
 - ▶ But the *records in a view are not stored in the DBMS*.
Rather, they are computed using a definition for the view

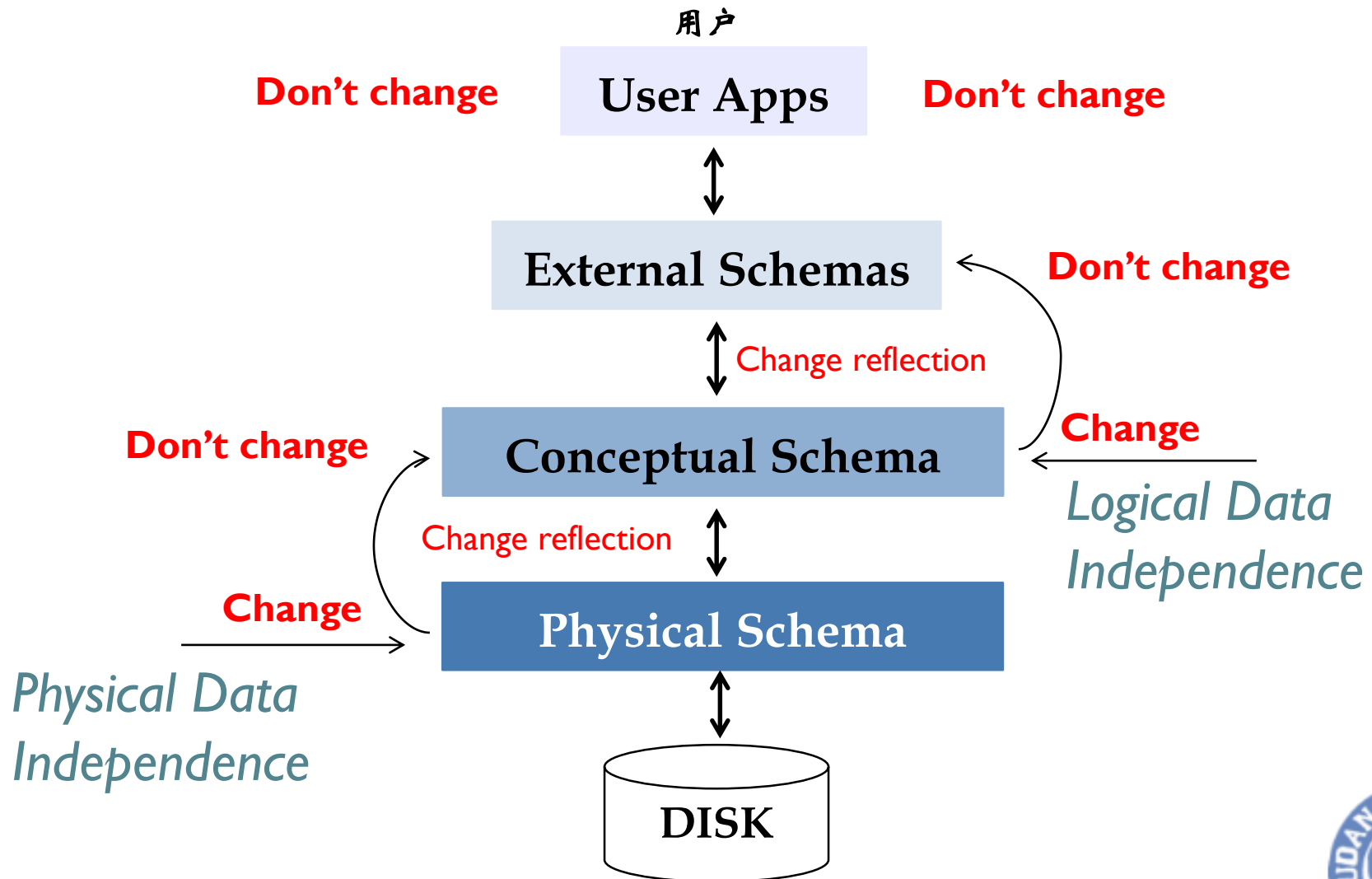


Data Independence

- ▶ Application programs are *insulated from changes* in the way the data is structured and stored
- ▶ **Logical data independence**
 - ▶ change of conceptual schemas without change of external views
- ▶ **Physical data independence**
 - ▶ change of physical database schemas without change of conceptual database schemas or application programs



Data Independence



Queries in a DBMS

- ▶ Consider the previous university database, a user may ask some questions:
 - ▶ *1. What is the name of the student with ID 1234567*
 - ▶ *2. How many students are enrolled in CS5647*
 - ▶ *3. What is the average salary of professors who teach course CS5647*
- ▶ The questions are called **queries**
- ▶ Queries can be posed by **query language**
- ▶ Relational model supports powerful query languages

```
SELECT name  
FROM Students S  
Where S.sid = '1234567'
```



Queries of Relational Model

- ▶ **Relational calculus** is a formal query language based on *mathematical logic*, and queries in this language have an *intuitive, precise meaning*
- ▶ **Relational algebra** based on a collection of *operators for manipulating relations*, which is equivalent in power to the calculus



Data Manipulation Language (DML)

- ▶ Language for *accessing and manipulating the data organized by the appropriate data model*
 - ▶ DML also known as **query language**
- ▶ Two classes of languages
 - ▶ **Procedural** – user specifies what data is required and how to get those data
 - ▶ **Declarative (nonprocedural)** – user specifies what data is required without specifying how to get those data
- ▶ **SQL** is the most widely used query language



Data Definition Language (DDL)

- ▶ Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- ▶ DDL compiler generates a set of table templates stored in a **data dictionary**
- ▶ Data dictionary contains **metadata** (i.e., data about data)
 - ▶ **Database schema**
 - ▶ **Integrity constraints**
 - ▶ Primary key (ID uniquely identifies instructors)
 - ▶ Referential integrity (**references** constraint in SQL)
 - e.g. *dept_name* value in any *instructor* tuple must appear in *department* relation
 - ▶ **Authorization**



SQL

- ▶ **SQL**: widely used non-procedural language

- ▶ Example: *Find the name of the instructor with ID 22222*

```
select  name
from    instructor
where   instructor.ID = '22222'
```

- ▶ Example: *Find the ID and building of instructors in the Physics dept.*

```
select  instructor.ID, department.building
from    instructor, department
where   instructor.dept_name = department.dept_name and
        department.dept_name = 'Physics'
```

- ▶ *Application programs generally access databases* through

- ▶ Language extensions to allow embedded SQL
 - ▶ Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database



Database Design

The process of designing the general structure of the database:

- ▶ **Conceptual Design** – *deciding on the conceptual schema of the database*
 - ▶ Providing the database designer with a **conceptual framework** in which to specify the data requirements of the database users, and how the database will be structured to fulfill these requirements.
 - ▶ A specification of **user requirements**
 - ▶ In terms of the relational model, the conceptual-design process involves decisions on **what attributes** we want to capture in the database and **how to group these attributes** to form the various tables.
 - ▶ A fully developed conceptual schema indicates the **functional requirements of the enterprise**.



Database Design

- ▶ **Logical Design** – *deciding on the database schema (database design requires that we find a “good” collection of relation schemas)*
 - ▶ **Business decision** – What attributes should we record in the database?
 - ▶ **Computer Science decision** – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- ▶ **Physical Design** – *deciding on the physical layout of the database*



Design Approaches

- ▶ **Normalization Theory** (Chapter 8)
 - ▶ Formalize what designs are bad, and test for them
- ▶ **Entity Relationship Model** (Chapter 7)
 - ▶ Models an enterprise as a collection of *entities* and *relationships*
 - ▶ **Entity**: a “*thing*” or “*object*” in the enterprise that is distinguishable from other objects described by a set of *attributes*
 - ▶ **Relationship**: an association among several entities
 - ▶ Represented diagrammatically by an *entity-relationship diagram*



Transaction Management

- ▶ Situation#1: Consider a database that holds information about **airline reservations**
 - ▶ Several travel agents are looking up information about available seats on various flights and making new seat reservations
 - ▶ When several users access (and possibly modify) a database concurrently

The DBMS must order their requests carefully to avoid conflicts !



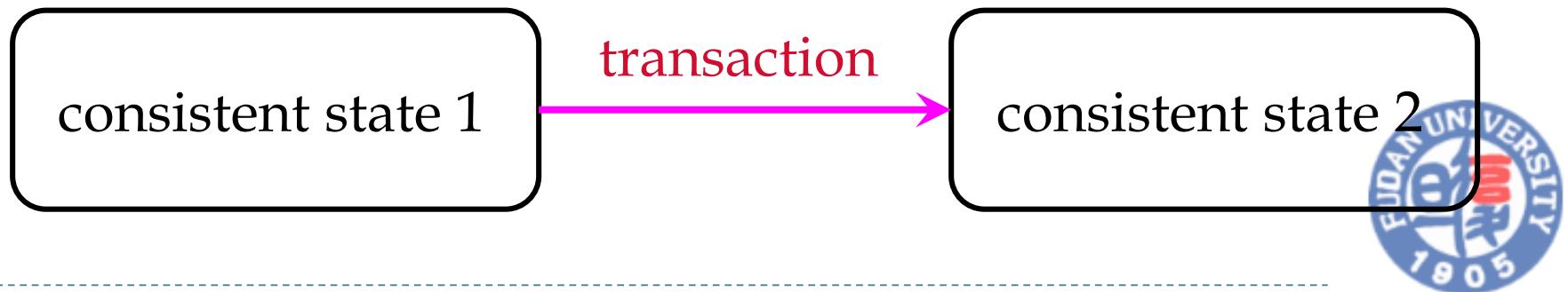
Transaction Management

- ▶ Situation#2: The Database system **Crashes**
 - ▶ If a travel agent asks for a reservation to be made, and the DBMS responds saying that the **reservation has been made**, the reservation should not be lost if the system crashes
 - ▶ If the DBMS has **not yet responded to the request**, but is in the process of making the necessary changes to the data while the crash occurs, the partial changes should be **undone** when the system comes back up



Key concept: Transaction

- ▶ A **transaction** is any one execution of a user program in a DBMS
 - ▶ An *atomic sequence* of database actions (reads/writes)
 - ▶ *Partial transactions* are not allowed
 - ▶ The effect of a group of transactions is equivalent to some serial execution of all transactions
- ▶ Takes DB from one **consistent state** to another



Key Properties: Transaction

- ▶ **Atomicity** (All-or-none requirement)
 - ▶ The operations in a transaction must happen in its entirety or not at all.
- ▶ **Consistency** (Correctness requirement)
 - ▶ The execution of a transaction preserve the consistency of the database.
- ▶ **Durability** (Persistence requirement):
 - ▶ After the successful execution of a transaction , the new changes of the database must persist, despite the possibility of system failure.



Concurrent Execution of Transactions

- ▶ A database is typically **shared by a large number of users**, who submit their requests to the DBMS independently
- ▶ A DBMS allows users to think of their programs **as if they were executing in isolation**, one after the other in some order chosen by the DBMS



Example of Concurrency

- ▶ If a program that **deposits cash** into an account is submitted to the DBMS
- ▶ At the same time another program that **debits money** from the same account
- ▶ Either of these programs could be run first by the DBMS, but their steps will **not be interleaved** in such a way that they interfere with each other



How to Deal with Concurrency

▶ **Locking Protocol**

- ▶ ensure that even though actions of several transactions might be interleaved, the net effect is identical to executing all transactions **in some serial order**
- ▶ **Lock**: a mechanism used to control access to database objects
 - ▶ **shared locks** on an object can be held by two different transactions at the same time
 - ▶ **exclusive lock** on an object ensures that no other transactions hold any lock on this object



Incomplete Transactions and System Crashes

- ▶ **Transactions can be interrupted** before running to completion for a variety of reasons, e.g., a system crash
- ▶ DBMS must ensure that the *changes* made by incomplete transactions are *removed* from the database
 - ▶ For example, if the DBMS is in the middle of transferring money from account A to account B

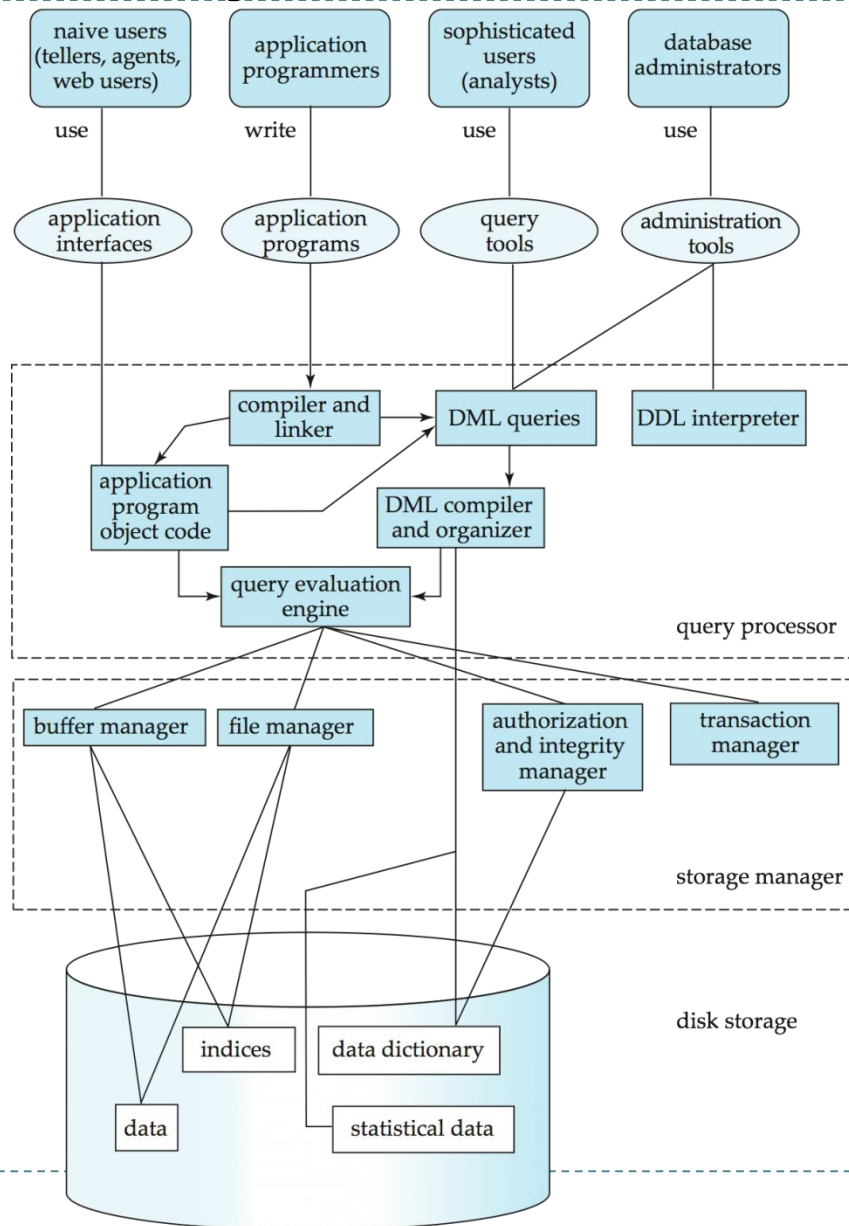


Incomplete Transactions and System Crashes

- ▶ DBMS maintains **a log of all writes** to the database
 - ▶ *each write action must be recorded* in the log (on disk) before the corresponding change is reflected in the database itself (**Write-Ahead Log or WAL**)
 - ▶ ensure that the changes made by a successfully *completed transaction are not lost* due to a system crash
- ▶ The time required to recover from a crash can be reduced by *periodically forcing* some information to disk; this periodic operation is called a *checkpoint*



Database System Internals



Database Architecture

The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:

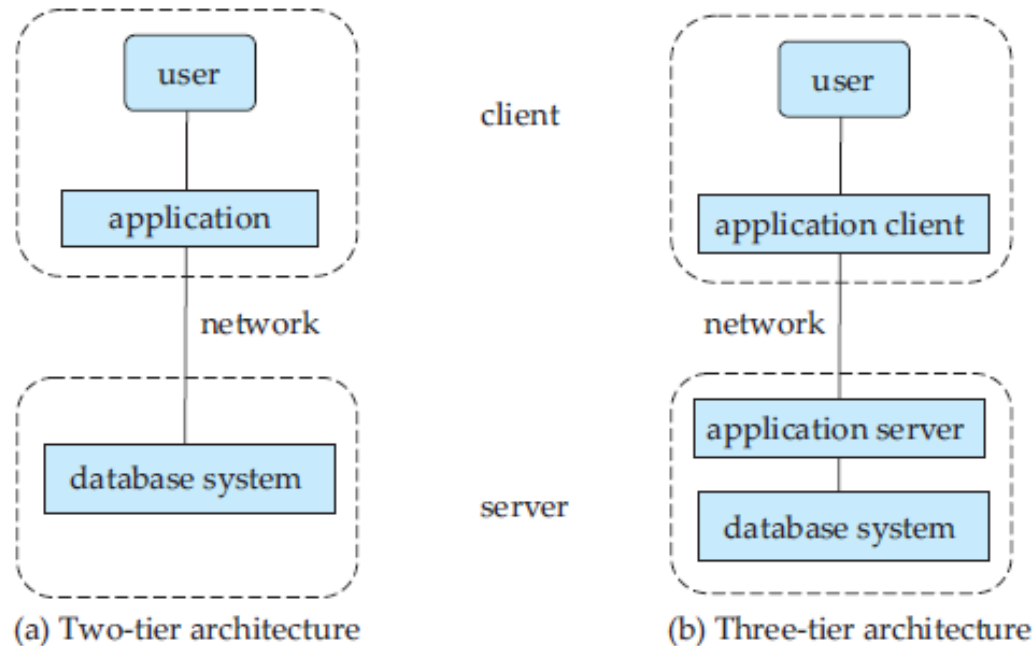
- ▶ **Centralized**
- ▶ **Client-server**
- ▶ **Parallel (multi-processor)**
- ▶ **Distributed**



Database Architecture

Database applications are usually partitioned into two or three parts:

- ▶ **Two-tier architecture**
- ▶ **Three-tier architecture**



Who Deal with Databases ?

- ▶ **Database implementors**

- ▶ Work for vendors such as Oracle and IBM

- ▶ **End users**

- ▶ Many *end users* simply use applications written by database application programmers and so require little technical knowledge about DBMS software
 - ▶ *Sophisticated users* who make more extensive use of a DBMS, such as writing their own queries, require a deeper understanding of its features



Who Deal with Databases ?

▶ **Application Programmers**

- ▶ Those who write programs (Cobol, C, Java) with embed DML calls or develop packages with software tools provided by the DBMS vendor

▶ **Database Administrator (DBA)** : has central control over the database and responsible for the following tasks:

- ▶ Schema definition/modification
- ▶ Storage structure definition/modification
- ▶ Authorization of data access
- ▶ Integrity constraints specification
- ▶ Monitoring performance
- ▶ Database tuning
- ▶ Database Recovery



Who Deal with Databases ?

